

Práctica Arduino

Funciones, punteros, arreglos y control de un LED

Índice general

1. Práctica Arduino con funciones, punteros y arreglos	1
1.1. Qué se construirá	1
1.2. Arduino, C y C++	1
1.3. Materiales y conexiones	2
1.3.1. Materiales	2
1.3.2. Conexiones del potenciómetro	2
1.3.3. Conexiones del LED	3
1.3.4. Diagrama conceptual del montaje	3
1.4. Primera etapa: lectura simple del potenciómetro	3
1.4.1. Lectura del código	4
1.5. Segunda etapa: separar el programa en funciones	4
1.5.1. Qué aporta esta etapa	6
1.6. Tercera etapa: una función que entrega varios resultados usando punteros	6
1.6.1. Lectura detallada de la llamada	8
1.6.2. Control básico del LED usando otra función con puntero	9
1.7. Cuarta etapa: agregar un arreglo de lecturas	11
1.7.1. Por qué el arreglo se relaciona con punteros	11
1.7.2. Función para llenar el arreglo	11
1.7.3. Función para analizar el arreglo	12
1.8. Código con arreglo de lecturas	12
1.8.1. Lectura detallada de la función <code>analizarLecturas</code>	14

1.9. Código final de la práctica	15
1.10. Explicación del código final	20
1.10.1. Constantes del programa	20
1.10.2. El arreglo <code>lecturas</code>	20
1.10.3. Arreglo como parámetro de función	20
1.10.4. Punteros como salidas múltiples	21
1.10.5. Por qué se usa <code>long</code> en algunas operaciones	21
1.11. Errores comunes en esta práctica	22
1.11.1. Olvidar la resistencia del LED	22
1.11.2. Conectar mal el terminal central del potenciómetro	22
1.11.3. Usar un puntero sin verificarlo	22
1.11.4. Recorrer más posiciones de las que tiene el arreglo	22
1.11.5. Confundir <code>p</code> con <code>*p</code>	23
1.12. Entrega sugerida	23
1.12.1. Cambios pequeños permitidos	23
1.13. Cierre conceptual	24

Capítulo 1

Práctica Arduino con funciones, punteros y arreglos

1.1. Qué se construirá

Se construirá un programa para Arduino que lee un potenciómetro, guarda varias lecturas en un arreglo, calcula el promedio, el valor mínimo y el valor máximo, y usa esos resultados para controlar un LED.

La práctica se desarrolla de manera progresiva. Primero se revisa una lectura simple del potenciómetro. Luego se separa el programa en funciones. Después se usan punteros para que una función pueda escribir varios resultados. Finalmente se agrega un arreglo pequeño de lecturas para relacionar arreglos, funciones y punteros.

Idea central

La idea principal es usar Arduino como laboratorio visible para estudiar esta relación:

funciones + punteros + arreglos = programas mejor organizados

El montaje es sencillo: un potenciómetro actúa como entrada analógica y un LED actúa como salida visual. El Monitor Serial permite observar los valores calculados por el programa.

1.2. Arduino, C y C++

El entorno de Arduino permite escribir programas con una sintaxis muy cercana a C, pero el compilador usado por Arduino trabaja realmente con C++. Por eso, en esta práctica se verán instrucciones muy familiares para C, como punteros, arreglos, funciones y operadores `&` y `*`, dentro de un archivo de Arduino con extensión `.ino`.

Clave de lectura

En Arduino, las ideas de punteros estudiadas en C siguen siendo válidas: un puntero guarda una dirección, `&x` obtiene la dirección de `x`, y `*p` permite acceder al contenido apuntado por `p`.

En C++ moderno se recomienda usar `nullptr` para representar un puntero que no apunta todavía a una dirección válida. En esta práctica se usará `nullptr` cuando se necesite verificar punteros.

1.3. Materiales y conexiones

1.3.1. Materiales

- Arduino UNO o placa compatible.
- Un potenciómetro de $10\text{ k}\Omega$.
- Un LED.
- Una resistencia de $220\ \Omega$ o $330\ \Omega$ para el LED.
- Protoboard.
- Cables de conexión.
- Cable USB para programar la placa.

1.3.2. Conexiones del potenciómetro

El potenciómetro tiene tres terminales. Los extremos se conectan a alimentación y tierra, y el terminal central entrega una señal variable.

Terminal del potenciómetro	Conexión
Un extremo	5V del Arduino
Otro extremo	GND del Arduino
Terminal central	Pin analógico A0

1.3.3. Conexiones del LED

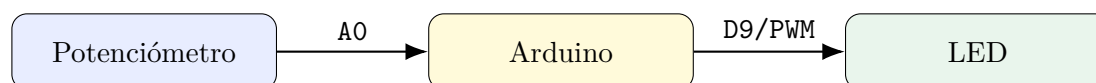
Elemento	Conexión
Ánodo del LED, pata larga	Pin digital 9, pasando por una resistencia
Cátodo del LED, pata corta	GND

El pin 9 se eligió porque en Arduino UNO permite salida PWM mediante `analogWrite`. Esto permitirá variar el brillo del LED según el promedio de las lecturas del potenciómetro.

Cuidado

No se debe conectar el LED directamente al pin sin resistencia. La resistencia limita la corriente y protege tanto el LED como el pin del Arduino.

1.3.4. Diagrama conceptual del montaje



El programa lee una señal analógica, calcula información y controla una salida.

1.4. Primera etapa: lectura simple del potenciómetro

Antes de usar punteros y arreglos, conviene verificar que el montaje funciona. El siguiente programa lee el potenciómetro y muestra el valor en el Monitor Serial.

Nombre del archivo: `etapa_01_lectura_simple_potenciometro.ino`

```

/*
  Etapa 1
  Lectura simple del potenciómetro conectado al pin A0.
  Este programa todavía no usa funciones propias, punteros ni arreglos.
*/

const int PIN_POTENCIOMETRO = A0;

void setup() {
  /* Iniciamos la comunicación serial a 9600 baudios. */
  Serial.begin(9600);
}
    
```

```

}

void loop() {
    /* Leemos el valor analógico del potenciómetro.
       En Arduino UNO, analogRead entrega un valor entre 0 y 1023. */
    int valorADC = analogRead(PIN_POTENCIOMETRO);

    /* Mostramos el valor leído. */
    Serial.print("ADC = ");
    Serial.println(valorADC);

    /* Pausa corta para no saturar el Monitor Serial. */
    delay(500);
}

```

1.4.1. Lectura del código

La constante PIN_POTENCIOMETRO guarda el pin usado para leer el potenciómetro. En Arduino UNO, `analogRead(A0)` convierte el voltaje del pin A0 en un número entre 0 y 1023.

Si el potenciómetro está cerca de GND, la lectura se aproxima a 0. Si está cerca de 5V, la lectura se aproxima a 1023.

Para comprobar

Abrir el Monitor Serial, girar lentamente el potenciómetro y verificar que el valor cambia. Si el valor no cambia, revisar las conexiones del terminal central del potenciómetro.

1.5. Segunda etapa: separar el programa en funciones

Una función permite encapsular una tarea. En Arduino ya existen dos funciones especiales: `setup()` y `loop()`. También se pueden crear funciones propias para que el programa quede más organizado.

En esta etapa se agrega una función para leer el potenciómetro y otra para mostrar la información.

Nombre del archivo: `etapa_02_funciones_basicas.ino`

```

/*
   Etapa 2
   Lectura del potenciómetro usando funciones propias.
   Todavía no usamos punteros. Cada función realiza una tarea pequeña.
   */

```

```
const int PIN_POTENCIOMETRO = A0;
const int ADC_MAX = 1023;
const float VREF = 5.0;

int leerADC(int pin) {
    /* Esta función recibe un pin analógico y devuelve la lectura ADC. */
    int valor = analogRead(pin);
    return valor;
}

float calcularVoltaje(int valorADC) {
    /* Convertimos el valor ADC a voltaje aproximado.
       Se usa VREF = 5.0 porque Arduino UNO trabaja normalmente con 5 V. */
    float voltaje = (valorADC * VREF) / ADC_MAX;
    return voltaje;
}

int calcularPorcentaje(int valorADC) {
    /* Convertimos el valor ADC a porcentaje.
       Se usa long para evitar desbordamiento en placas donde int es de 16 bits. */
    int porcentaje = (int)((long)valorADC * 100L / ADC_MAX);
    return porcentaje;
}

void mostrarDatos(int valorADC, float voltaje, int porcentaje) {
    /* Mostramos los datos en una sola línea del Monitor Serial. */
    Serial.print("ADC = ");
    Serial.print(valorADC);

    Serial.print(" | Voltaje = ");
    Serial.print(voltaje, 2);
    Serial.print(" V");

    Serial.print(" | Porcentaje = ");
    Serial.print(porcentaje);
    Serial.println(" %");
}

void setup() {
    /* Iniciamos el Monitor Serial. */
    Serial.begin(9600);
}

void loop() {
    /* Leemos el potenciómetro. */
    int valorADC = leerADC(PIN_POTENCIOMETRO);

    /* Calculamos el voltaje. */

```



```
float voltaje = calcularVoltaje(valorADC);

/* Calculamos el porcentaje. */
int porcentaje = calcularPorcentaje(valorADC);

/* Mostramos todo por serial. */
mostrarDatos(valorADC, voltaje, porcentaje);

delay(500);
}
```

1.5.1. Qué aporta esta etapa

El programa ya no está concentrado completamente en `loop()`. Cada función tiene una responsabilidad clara:

- `leerADC`: obtiene el valor del pin analógico;
- `calcularVoltaje`: convierte la lectura ADC a voltaje;
- `calcularPorcentaje`: convierte la lectura ADC a porcentaje;
- `mostrarDatos`: imprime la información en el Monitor Serial.

Clave de lectura

Una función ayuda a organizar el programa. En lugar de escribir todo en `loop()`, se divide el problema en tareas pequeñas.

1.6. Tercera etapa: una función que entrega varios resultados usando punteros

En la etapa anterior se usaron varias funciones, pero cada función devolvía un solo valor con `return`. Ahora se construye una función que obtiene tres resultados al mismo tiempo: lectura ADC, voltaje y porcentaje.

Para lograrlo, la función recibe direcciones. Esas direcciones se usan como canales de salida.

Idea central

Una función puede recibir punteros para escribir varios resultados en variables externas. Esta es una de las razones más útiles para estudiar punteros.

La función tendrá esta forma:

```
void leerPotenciometro(int pin, int *valorADC, float *voltaje, int *porcentaje)
```

Se lee así:

- pin: pin analógico que se desea leer;
- int *valorADC: dirección donde se escribirá la lectura ADC;
- float *voltaje: dirección donde se escribirá el voltaje;
- int *porcentaje: dirección donde se escribirá el porcentaje.

Nombre del archivo: etapa_03_funcion_con_punteros.ino

```
/*
  Etapa 3
  Una función entrega varios resultados usando punteros.
  Se calculan valor ADC, voltaje y porcentaje en una sola función.
*/

const int PIN_POTENCIOMETRO = A0;
const int ADC_MAX = 1023;
const float VREF = 5.0;

void leerPotenciometro(int pin, int *valorADC, float *voltaje, int *porcentaje) {
  /* Verificamos que las direcciones recibidas sean válidas.
     Si algún puntero es nullptr, no se debe usar * sobre él. */
  if (valorADC == nullptr || voltaje == nullptr || porcentaje == nullptr) {
    return;
  }

  /* Escribimos la lectura ADC en la variable externa apuntada por valorADC. */
  *valorADC = analogRead(pin);

  /* Usamos *valorADC porque allí quedó guardada la lectura real. */
  *voltaje = (*valorADC * VREF) / ADC_MAX;

  /* Calculamos el porcentaje.
     Se usa long para evitar desbordamiento en la multiplicación. */
  *porcentaje = (int)((long)(*valorADC) * 100L / ADC_MAX);
}

void mostrarDatos(int valorADC, float voltaje, int porcentaje) {
  /* Mostramos los resultados calculados. */
  Serial.print("ADC = ");
  Serial.print(valorADC);

  Serial.print(" | Voltaje = ");
  Serial.print(voltaje, 2);
  Serial.print(" V");
}
```

```

    Serial.print(" | Porcentaje = ");
    Serial.print(porcentaje);
    Serial.println(" %");
}

void setup() {
    Serial.begin(9600);
}

void loop() {
    /* Variables donde se guardarán los resultados. */
    int valorADC = 0;
    float voltaje = 0.0;
    int porcentaje = 0;

    /* Enviamos direcciones para que la función escriba los resultados. */
    leerPotenciometro(PIN_POTENCIOMETRO, &valorADC, &voltaje, &porcentaje);

    /* Mostramos los valores ya calculados. */
    mostrarDatos(valorADC, voltaje, porcentaje);

    delay(500);
}

```

1.6.1. Lectura detallada de la llamada

La llamada principal es:

```
leerPotenciometro(PIN_POTENCIOMETRO, &valorADC, &voltaje, &porcentaje);
```

Aquí se envían cuatro argumentos:

- PIN_POTENCIOMETRO: el pin que se leerá;
- &valorADC: dirección de la variable donde se guardará la lectura;
- &voltaje: dirección de la variable donde se guardará el voltaje;
- &porcentaje: dirección de la variable donde se guardará el porcentaje.

Dentro de la función, las expresiones `*valorADC`, `*voltaje` y `*porcentaje` permiten escribir en las variables originales declaradas en `loop()`.

Comparación

`valorADC` dentro de la función es una dirección.
`*valorADC` es el contenido ubicado en esa dirección.
`&valorADC` en `loop()` es la dirección de la variable local `valorADC`.

1.6.2. Control básico del LED usando otra función con puntero

Ahora se agrega un LED. La función `decidirEstadoLED` recibe el porcentaje y escribe el estado del LED en una variable externa.

Nombre del archivo: `etapa_04_potenciometro_led_punteros.ino`

```
/*
  Etapa 4
  Potenciómetro + LED.
  Una función lee el potenciómetro y otra decide el estado del LED.
*/

const int PIN_POTENCIOMETRO = A0;
const int PIN_LED = 9;
const int ADC_MAX = 1023;
const float VREF = 5.0;
const int UMBRAL_LED = 50;

void leerPotenciometro(int pin, int *valorADC, float *voltaje, int *porcentaje) {
  /* Verificamos punteros antes de escribir resultados. */
  if (valorADC == nullptr || voltaje == nullptr || porcentaje == nullptr) {
    return;
  }

  /* Se escribe la lectura ADC en la variable apuntada. */
  *valorADC = analogRead(pin);

  /* Se escribe el voltaje calculado en la variable apuntada. */
  *voltaje = (*valorADC * VREF) / ADC_MAX;

  /* Se escribe el porcentaje calculado en la variable apuntada. */
  *porcentaje = (int)((long)(*valorADC) * 100L / ADC_MAX);
}

void decidirEstadoLED(int porcentaje, int umbral, int *estadoLED) {
  /* Verificamos que el puntero estadoLED sea válido. */
  if (estadoLED == nullptr) {
    return;
  }

  /* Si el porcentaje supera o iguala el umbral, el LED se encenderá. */
  if (porcentaje >= umbral) {
    *estadoLED = HIGH;
  } else {
    *estadoLED = LOW;
  }
}
```

```

void aplicarEstadoLED(int pinLED, int estadoLED) {
    /* Aplicamos físicamente el estado al pin del LED. */
    digitalWrite(pinLED, estadoLED);
}

void mostrarDatos(int valorADC, float voltaje, int porcentaje, int estadoLED) {
    /* Mostramos todos los datos en el Monitor Serial. */
    Serial.print("ADC = ");
    Serial.print(valorADC);

    Serial.print(" | Voltaje = ");
    Serial.print(voltaje, 2);
    Serial.print(" V");

    Serial.print(" | Porcentaje = ");
    Serial.print(porcentaje);
    Serial.print(" %");

    Serial.print(" | LED = ");
    if (estadoLED == HIGH) {
        Serial.println("ENCENDIDO");
    } else {
        Serial.println("APAGADO");
    }
}

void setup() {
    Serial.begin(9600);

    /* Configuramos el pin del LED como salida. */
    pinMode(PIN_LED, OUTPUT);
}

void loop() {
    int valorADC = 0;
    float voltaje = 0.0;
    int porcentaje = 0;
    int estadoLED = LOW;

    /* La función escribe tres resultados usando punteros. */
    leerPotenciometro(PIN_POTENCIOMETRO, &valorADC, &voltaje, &porcentaje);

    /* La función escribe el estado del LED usando un puntero. */
    decidirEstadoLED(porcentaje, UMBRAL_LED, &estadoLED);

    /* Se aplica el estado calculado al LED. */
    aplicarEstadoLED(PIN_LED, estadoLED);
}

```

```

    /* Se muestra el reporte. */
    mostrarDatos(valorADC, voltaje, porcentaje, estadoLED);

    delay(500);
}

```

1.7. Cuarta etapa: agregar un arreglo de lecturas

Una sola lectura del potenciómetro puede variar ligeramente. Para obtener una medida más estable, se pueden tomar varias lecturas y calcular un promedio.

Para ello se usará un arreglo pequeño:

```
int lecturas[10];
```

Ese arreglo puede guardar diez valores enteros. Cada posición del arreglo almacena una lectura ADC.

Clave de lectura

Un arreglo permite guardar varios datos del mismo tipo bajo un solo nombre. En esta práctica, el arreglo guardará varias lecturas del potenciómetro.

1.7.1. Por qué el arreglo se relaciona con punteros

Cuando se pasa un arreglo a una función, normalmente la función recibe la dirección del primer elemento del arreglo. Por eso, en una función como:

```
void tomarLecturas(int pin, int datos[], int cantidad)
```

el parámetro `datos` permite acceder al arreglo original. Si la función escribe en `datos[i]`, está modificando el arreglo que se envió desde `loop()`.

Comparación

En muchos contextos de C y C++, el nombre de un arreglo se puede usar como dirección del primer elemento. Por eso `datos[i]` y `*(datos + i)` están muy relacionados. Para esta práctica se usará `datos[i]` porque es más legible al comenzar.

1.7.2. Función para llenar el arreglo

La función `tomarLecturas` recibe el pin, el arreglo, la cantidad de lecturas y una pausa entre lecturas.

```
void tomarLecturas(int pin, int datos[], int cantidad, int pausaMs)
```

La función no devuelve un valor con `return`. Su tarea es escribir dentro del arreglo recibido.

1.7.3. Función para analizar el arreglo

Después de llenar el arreglo, otra función calcula promedio, mínimo y máximo. Esa función recibe el arreglo como entrada y recibe punteros para escribir los resultados.

```
void analizarLecturas(int datos[], int cantidad,
                    int *promedio,
                    int *minimo,
                    int *maximo)
```

Esta función combina las dos ideas principales de la práctica:

- el arreglo entrega varias lecturas de entrada;
- los punteros permiten escribir varios resultados de salida.

1.8. Código con arreglo de lecturas

Nombre del archivo: `etapa_05_arreglo_lecturas_punteros.ino`

```
/*
  Etapa 5
  Se toman varias lecturas del potenciómetro y se guardan en un arreglo.
  Luego se calcula promedio, mínimo y máximo usando punteros de salida.
*/

const int PIN_POTENCIOMETRO = A0;
const int CANTIDAD_LECTURAS = 10;
const int PAUSA_ENTRE_LECTURAS_MS = 20;

void tomarLecturas(int pin, int datos[], int cantidad, int pausaMs) {
  /* Verificamos que el arreglo no sea nullptr y que la cantidad sea válida. */
  if (datos == nullptr || cantidad <= 0) {
    return;
  }

  /* Recorremos el arreglo y guardamos una lectura en cada posición. */
  for (int i = 0; i < cantidad; i++) {
    datos[i] = analogRead(pin);
    delay(pausaMs);
  }
}
```

```

    }
}

void analizarLecturas(int datos[], int cantidad,
                     int *promedio,
                     int *minimo,
                     int *maximo) {
    /* Verificamos punteros y cantidad antes de procesar. */
    if (datos == nullptr || promedio == nullptr || minimo == nullptr || maximo ==
        nullptr || cantidad <= 0) {
        return;
    }

    /* Usamos long para acumular sin riesgo de desbordamiento. */
    long suma = 0;

    /* Inicializamos mínimo y máximo con el primer dato del arreglo. */
    *minimo = datos[0];
    *maximo = datos[0];

    /* Recorremos todas las lecturas. */
    for (int i = 0; i < cantidad; i++) {
        /* Acumulamos la lectura actual. */
        suma = suma + datos[i];

        /* Actualizamos el mínimo si encontramos un valor menor. */
        if (datos[i] < *minimo) {
            *minimo = datos[i];
        }

        /* Actualizamos el máximo si encontramos un valor mayor. */
        if (datos[i] > *maximo) {
            *maximo = datos[i];
        }
    }

    /* Calculamos el promedio entero. */
    *promedio = (int)(suma / cantidad);
}

void mostrarArreglo(int datos[], int cantidad) {
    /* Verificamos que el arreglo sea válido. */
    if (datos == nullptr || cantidad <= 0) {
        return;
    }

    Serial.print("Lecturas: ");

```



```

    /* Mostramos cada lectura del arreglo. */
    for (int i = 0; i < cantidad; i++) {
        Serial.print(datos[i]);
        Serial.print(" ");
    }

    Serial.println();
}

void setup() {
    Serial.begin(9600);
}

void loop() {
    /* Arreglo local para guardar varias lecturas. */
    int lecturas[CANTIDAD_LECTURAS];

    /* Variables donde se guardarán los resultados. */
    int promedio = 0;
    int minimo = 0;
    int maximo = 0;

    /* Llenamos el arreglo con lecturas del potenciómetro. */
    tomarLecturas(PIN_POTENCIOMETRO, lecturas, CANTIDAD_LECTURAS,
    PAUSA_ENTRE_LECTURAS_MS);

    /* Analizamos el arreglo y escribimos los resultados mediante punteros. */
    analizarLecturas(lecturas, CANTIDAD_LECTURAS, &promedio, &minimo, &maximo);

    /* Mostramos el arreglo y sus resultados. */
    mostrarArreglo(lecturas, CANTIDAD_LECTURAS);
    Serial.print("Promedio = ");
    Serial.println(promedio);
    Serial.print("Minimo   = ");
    Serial.println(minimo);
    Serial.print("Maximo   = ");
    Serial.println(maximo);
    Serial.println("-----");

    delay(1000);
}

```

1.8.1. Lectura detallada de la función analizarLecturas

La función recibe:

```
int datos[]
```

Esto permite acceder a las lecturas tomadas. También recibe:

```
int *promedio, int *minimo, int *maximo
```

Estos punteros son canales de salida. La función escribe en *promedio, *minimo y *maximo. Por eso, después de llamar la función, las variables de loop() ya contienen los resultados.

Clave de lectura

El arreglo lleva los datos hacia la función. Los punteros permiten que la función devuelva varios resultados hacia afuera.

1.9. Código final de la práctica

El código final toma diez lecturas del potenciómetro, calcula promedio, mínimo y máximo, convierte el promedio a voltaje, porcentaje y brillo PWM, y controla el LED según el valor promedio.

Nombre del archivo: practica_final_potenciometro_led_arreglo_punteros.ino

```
/*
  Práctica final
  Arduino + potenciómetro + LED + funciones + punteros + arreglo.

  El programa realiza lo siguiente:
  1. Toma varias lecturas del potenciómetro.
  2. Guarda las lecturas en un arreglo.
  3. Calcula promedio, mínimo y máximo.
  4. Convierte el promedio a voltaje, porcentaje y brillo PWM.
  5. Controla el LED según el porcentaje promedio.
  6. Muestra los resultados por el Monitor Serial.
*/

const int PIN_POTENCIOMETRO = A0;
const int PIN_LED = 9;

const int ADC_MAX = 1023;
const int PWM_MAX = 255;
const float VREF = 5.0;

const int CANTIDAD_LLECTURAS = 10;
const int PAUSA_ENTRE_LLECTURAS_MS = 20;
const int PAUSA_CICLO_MS = 1000;
const int UMBRAL_LED = 50;
```

```

void tomarLecturas(int pin, int datos[], int cantidad, int pausaMs) {
    /* Verificamos que el arreglo sea válido y que la cantidad tenga sentido. */
    if (datos == nullptr || cantidad <= 0) {
        return;
    }

    /* Guardamos una lectura ADC en cada posición del arreglo. */
    for (int i = 0; i < cantidad; i++) {
        datos[i] = analogRead(pin);
        delay(pausaMs);
    }
}

void analizarLecturas(int datos[], int cantidad,
                     int *promedio,
                     int *minimo,
                     int *maximo) {
    /* Verificamos que las direcciones recibidas sean válidas. */
    if (datos == nullptr || promedio == nullptr || minimo == nullptr || maximo ==
    nullptr || cantidad <= 0) {
        return;
    }

    /* La suma puede crecer, por eso usamos long. */
    long suma = 0;

    /* Tomamos el primer dato como referencia inicial. */
    *minimo = datos[0];
    *maximo = datos[0];

    /* Recorremos todas las lecturas. */
    for (int i = 0; i < cantidad; i++) {
        /* Acumulamos la lectura actual. */
        suma = suma + datos[i];

        /* Si encontramos una lectura menor, actualizamos el mínimo. */
        if (datos[i] < *minimo) {
            *minimo = datos[i];
        }

        /* Si encontramos una lectura mayor, actualizamos el máximo. */
        if (datos[i] > *maximo) {
            *maximo = datos[i];
        }
    }

    /* Guardamos el promedio en la variable externa apuntada por promedio. */
    *promedio = (int)(suma / cantidad);
}

```

```

}

void convertirPromedio(int promedioADC,
                      float *voltaje,
                      int *porcentaje,
                      int *brilloPWM) {
    /* Verificamos que los punteros de salida sean válidos. */
    if (voltaje == nullptr || porcentaje == nullptr || brilloPWM == nullptr) {
        return;
    }

    /* Convertimos ADC a voltaje aproximado. */
    *voltaje = (promedioADC * VREF) / ADC_MAX;

    /* Convertimos ADC a porcentaje usando long para evitar desbordamiento. */
    *porcentaje = (int)((long)promedioADC * 100L / ADC_MAX);

    /* Convertimos ADC a valor PWM entre 0 y 255. */
    *brilloPWM = (int)((long)promedioADC * PWM_MAX / ADC_MAX);
}

void decidirEstadoLED(int porcentaje, int umbral, int *estadoLED) {
    /* Verificamos que el puntero de salida sea válido. */
    if (estadoLED == nullptr) {
        return;
    }

    /* El LED se considera activo si el porcentaje supera el umbral. */
    if (porcentaje >= umbral) {
        *estadoLED = HIGH;
    } else {
        *estadoLED = LOW;
    }
}

void aplicarSalidaLED(int pinLED, int estadoLED, int brilloPWM) {
    /* Si el estado es LOW, apagamos el LED. */
    if (estadoLED == LOW) {
        analogWrite(pinLED, 0);
    } else {
        /* Si el estado es HIGH, aplicamos el brillo calculado. */
        analogWrite(pinLED, brilloPWM);
    }
}

void mostrarLecturas(int datos[], int cantidad) {
    /* Verificamos que el arreglo sea válido. */
    if (datos == nullptr || cantidad <= 0) {

```

```

        return;
    }

    Serial.print("Lecturas: ");

    /* Imprimimos todos los valores del arreglo. */
    for (int i = 0; i < cantidad; i++) {
        Serial.print(datos[i]);
        Serial.print(" ");
    }

    Serial.println();
}

void mostrarReporte(int promedio, int minimo, int maximo,
                    float voltaje, int porcentaje,
                    int brilloPWM, int estadoLED) {
    /* Mostramos un reporte completo del ciclo actual. */
    Serial.print("Promedio ADC = ");
    Serial.println(promedio);

    Serial.print("Minimo ADC   = ");
    Serial.println(minimo);

    Serial.print("Maximo ADC   = ");
    Serial.println(maximo);

    Serial.print("Voltaje      = ");
    Serial.print(voltaje, 2);
    Serial.println(" V");

    Serial.print("Porcentaje   = ");
    Serial.print(porcentaje);
    Serial.println(" %");

    Serial.print("Brillo PWM   = ");
    Serial.println(brilloPWM);

    Serial.print("LED          = ");
    if (estadoLED == HIGH) {
        Serial.println("ENCENDIDO");
    } else {
        Serial.println("APAGADO");
    }
}

Serial.println("-----");
}

```

```

void setup() {
    /* Iniciamos comunicación serial. */
    Serial.begin(9600);

    /* Configuramos el pin del LED como salida. */
    pinMode(PIN_LED, OUTPUT);
}

void loop() {
    /* Arreglo local para almacenar varias lecturas ADC. */
    int lecturas[CANTIDAD_LECTURAS];

    /* Variables para resultados del análisis. */
    int promedio = 0;
    int minimo = 0;
    int maximo = 0;

    /* Variables para resultados convertidos. */
    float voltaje = 0.0;
    int porcentaje = 0;
    int brilloPWM = 0;

    /* Variable para el estado del LED. */
    int estadoLED = LOW;

    /* 1. Tomamos varias lecturas y las guardamos en el arreglo. */
    tomarLecturas(PIN_POTENCIOMETRO, lecturas, CANTIDAD_LECTURAS,
    PAUSA_ENTRE_LECTURAS_MS);

    /* 2. Analizamos el arreglo usando punteros de salida. */
    analizarLecturas(lecturas, CANTIDAD_LECTURAS, &promedio, &minimo, &maximo);

    /* 3. Convertimos el promedio a voltaje, porcentaje y PWM. */
    convertirPromedio(promedio, &voltaje, &porcentaje, &brilloPWM);

    /* 4. Decidimos si el LED debe encenderse o apagarse. */
    decidirEstadoLED(porcentaje, UMBRAL_LED, &estadoLED);

    /* 5. Aplicamos el estado y el brillo al LED. */
    aplicarSalidaLED(PIN_LED, estadoLED, brilloPWM);

    /* 6. Mostramos el arreglo y el reporte completo. */
    mostrarLecturas(lecturas, CANTIDAD_LECTURAS);
    mostrarReporte(promedio, minimo, maximo, voltaje, porcentaje, brilloPWM,
    estadoLED);

    delay(PAUSA_CICLO_MS);
}

```

1.10. Explicación del código final

1.10.1. Constantes del programa

Las constantes permiten cambiar valores importantes en un solo lugar. Por ejemplo:

```
const int CANTIDAD_LECTURAS = 10;
```

indica que el arreglo tendrá diez lecturas. Si después se desea trabajar con veinte lecturas, se puede cambiar esta constante a 20. El resto del programa seguirá usando ese valor.

Cuidado

En Arduino UNO la memoria RAM es limitada. Por eso se recomienda usar arreglos pequeños al comenzar. Diez lecturas de tipo `int` ocupan poca memoria y son suficientes para esta práctica.

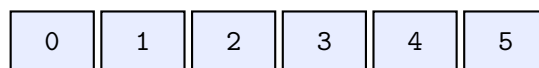
1.10.2. El arreglo `lecturas`

Dentro de `loop()` aparece:

```
int lecturas[CANTIDAD_LECTURAS];
```

Esto crea un arreglo de enteros. Cada posición del arreglo guardará una lectura ADC del potenciómetro.

Conceptualmente:



primeras posiciones del arreglo `lecturas`

Cada casilla guarda un valor como 0, 512, 850 o 1023, según la posición del potenciómetro.

1.10.3. Arreglo como parámetro de función

La función:

```
void tomarLecturas(int pin, int datos[], int cantidad, int pausaMs)
```

recibe `datos[]` como parámetro. En la llamada:

```
tomarLecturas(PIN_POTENCIOMETRO, lecturas, CANTIDAD_LECTURAS,
    PAUSA_ENTRE_LECTURAS_MS);
```

el nombre `lecturas` permite que la función acceda al arreglo original. Por eso, cuando la función escribe:

```
datos[i] = analogRead(pin);
```

está guardando datos dentro del arreglo `lecturas` declarado en `loop()`.

Clave de lectura

Cuando un arreglo se pasa a una función, la función puede modificar sus elementos. Por eso también se envía la cantidad de elementos, porque la función no debe adivinar hasta dónde puede recorrerlo.

1.10.4. Punteros como salidas múltiples

La función:

```
void analizarLecturas(int datos[], int cantidad,
                    int *promedio,
                    int *minimo,
                    int *maximo)
```

recibe un arreglo de entrada y tres punteros de salida. Esto permite que la función escriba tres resultados distintos:

```
*promedio = (int)(suma / cantidad);
*minimo = datos[0];
*maximo = datos[0];
```

Cuando se llama:

```
analizarLecturas(lecturas, CANTIDAD_LECTURAS, &promedio, &minimo, &maximo);
```

se envían las direcciones de las variables `promedio`, `minimo` y `maximo`. Así la función puede escribir en esas variables.

Comparación

`promedio` dentro de `loop()` es una variable entera.
`&promedio` es la dirección de esa variable.
`int *promedio` dentro de la función es un puntero que recibe esa dirección.
`*promedio` permite escribir en la variable original.

1.10.5. Por qué se usa `long` en algunas operaciones

En Arduino UNO, el tipo `int` suele ocupar 16 bits. Eso significa que puede quedarse corto en algunas multiplicaciones. Por ejemplo:


```
promedioADC * 255
```

podría superar el rango de un `int` si `promedioADC` está cerca de 1023. Para evitar problemas, se usa:

```
(long)promedioADC * PWM_MAX
```

Así la multiplicación se realiza usando un tipo más grande.

Cuidado

Este detalle es importante en microcontroladores. En un computador, `int` suele tener más capacidad; en Arduino UNO, `int` puede ser más pequeño.

1.11. Errores comunes en esta práctica

1.11.1. Olvidar la resistencia del LED

El LED debe tener resistencia. Sin resistencia, puede circular demasiada corriente.

1.11.2. Conectar mal el terminal central del potenciómetro

El terminal central del potenciómetro debe ir a A0. Si se conecta un extremo a A0, la lectura puede no comportarse como se espera.

1.11.3. Usar un puntero sin verificarlo

Antes de escribir mediante `*p`, conviene verificar que `p` no sea `nullptr`.

```
if (p != nullptr) {
    *p = 100;
}
```

1.11.4. Recorrer más posiciones de las que tiene el arreglo

Si el arreglo tiene diez posiciones, los índices válidos van de 0 a 9. No se debe acceder a `lecturas[10]`.

```
for (int i = 0; i < CANTIDAD_LECTURAS; i++) {
    /* i toma valores desde 0 hasta CANTIDAD_LECTURAS - 1. */
}
```

1.11.5. Confundir p con *p

Comparación

p es la dirección guardada en el puntero.
 *p es el contenido ubicado en esa dirección.
 &x es la dirección de la variable x.

1.12. Entrega sugerida

La entrega puede consistir en un solo archivo:

`practica_final_potenciometro_led_arreglo_punteros.ino`

El programa debe conservar estas características:

- lectura del potenciómetro por A0;
- control del LED por el pin 9;
- uso de funciones propias;
- uso de al menos un arreglo de lecturas;
- uso de punteros como parámetros de salida;
- cálculo de promedio, mínimo y máximo;
- reporte por Monitor Serial;
- comentarios suficientes en el código.

1.12.1. Cambios pequeños permitidos

Se pueden realizar cambios pequeños sin salirse de los temas trabajados:

- cambiar la cantidad de lecturas;
- cambiar el umbral de encendido del LED;
- cambiar el tiempo entre lecturas;
- mostrar más o menos información en el Monitor Serial;
- usar otro pin PWM para el LED, siempre que se actualice la constante correspondiente.

Cuidado

Para mantener la práctica acotada, no se requiere usar memoria dinámica, clases, objetos, interrupciones, sensores complejos ni comunicación inalámbrica.

1.13. Cierre conceptual

Esta práctica une varios conceptos básicos de programación con un montaje físico sencillo. El potenciómetro permite producir datos; el arreglo permite guardar varias lecturas; las funciones organizan el programa; los punteros permiten que una función escriba varios resultados; y el LED permite observar una respuesta del sistema.

Clave de lectura

La idea final puede resumirse así: el arreglo guarda varios datos, la función los procesa y los punteros permiten devolver varios resultados.

Arduino + funciones + arreglos + punteros = práctica visible y útil